Destructuring

Destructuring is a modern JavaScript feature (ES6) that lets you **extract values from arrays or objects and assign them to variables in a clean way**.

❌ Without Destructuring (Old Way)

```
const user = {
  name: "Prajwal",
  age: 22,
  city: "Mumbai"
};

const name = user.name;
const age = user.age;
const city = user.city;
```

Too repetitive. Hard to scale.

✅ With Destructuring

```
const { name, age, city } = user;
```

Same result. **Cleaner. Faster. Professional.**

2. Object Destructuring

Basic Syntax

```
const { key } = object;
```

Example:

```
const product = {
  title: "Laptop",
  price: 55000
};
```

```
const { title, price } = product;

console.log(title); // Laptop
console.log(price); // 55000
```

Rename Variables (Very Important in Backend)

```
const { title: productTitle } = product;

console.log(productTitle);
```

👉 Used when API response names are messy.

If value doesn't exist:

```
const { discount = 0 } = product;

console.log(discount); // 0
```

🎯 **Nested Object Destructuring**

Very common when working with APIs / MongoDB.

```
const order = {
  id: 1,
  customer: {
    name: "Prajwal",
    address: "Thane"
  }
};

const { customer: { name, address } } = order;

console.log(name);
```

3. Array Destructuring

Basic Syntax

```
const [a, b] = array;
```

Example:

```
const colors = ["red", "blue", "green"];

const [first, second] = colors;

console.log(first);  // red
console.log(second); // blue
```

🎯 **Skip Values**

```
const [ , , third] = colors;

console.log(third); // green
```

🎯 **Default Values**

```
const [a = "black"] = [];

console.log(a); // black
```

Swap Variables (Interview Favorite)

```
let x = 10;
let y = 20;

[x, y] = [y, x];

console.log(x, y); // 20, 10
```

Destructuring in Function Parameters (Used Everywhere)

```
function createUser({ name, age }) {
console.log(name, age);
}
```

Using Rest Operator with Destructuring

Collect remaining values:

```
const user = {
  name: "Prajwal",
  age: 22,
  city: "Mumbai"
};

const { name, ...rest } = user;

console.log(name); // Prajwal
console.log(rest); // { age: 22, city: "Mumbai" }
```

Destructuring with Arrays + Rest

```
const numbers = [10, 20, 30, 40];

const [first, ...remaining] = numbers;

console.log(first);      // 10
console.log(remaining);  // [20,30,40]
```

Destructuring from Function Return

```
function getStats() {
  return [100, 200];
}

const [views, likes] = getStats();
```

1. What will be the output?

```
const user = { name: "Prajwal", age: 22 };

const { name, city } = user;

console.log(city);
```

✅ **Answer:** undefined

👉 Because city does not exist.
Destructuring **does NOT throw error for missing keys**.

2. Default Value Trick

```
const user = { name: "Prajwal" };

const { name, city = "Mumbai" } = user;

console.log(city);
```

✅ **Answer:** "Mumbai"

👉 Default value is used **only when value is undefined**.

🔥 **3. Default Value Edge Case (Very Tricky)**

```
const user = { city: null };
```

```
const { city = "Mumbai" } = user;

console.log(city);
```

✅ **Answer:** null

⚠️ Default works only for undefined, not null.

Order vs Key Confusion Question

```
const obj = { a: 1, b: 2 };

const { b, a } = obj;

console.log(a, b);
```

✅ **Answer:** 1 2

👉 Object destructuring is **NOT order-based**
It is **key-based**.

. Array Destructuring IS Order-Based

```
const arr = [1, 2];

const [b, a] = arr;

console.log(a, b);
```

✅ **Answer:** 2 1

👉 Arrays depend on position.